

파이썬 보강 과제 규칙

1. 과제 규칙 (= 학습법)
- 규칙 1. 코드보다 먼저, 지문의 신호어를 찾는다
- 규칙 2. 설계 주석을 먼저 쓴다 — 단, "왜"까지 쓴다
- ✖

이렇게 쓰면 안됩니다
- ✔

이렇게 써야 합니다
- 왜 이렇게까지 해야할까?
- 금지 표현
- 규칙 3. 골격 4종은 백지에서 즉시 나와야 한다
- 규칙 4. 실행 전에 결과를 손으로 예측한다
- 규칙 5. 30분 넘게 막히면 무조건 질문한다
- 규칙 6. 이해가 안 돼도 일단 넘어간다
- 규칙 7. 문제를 하나 변형해서 고민해본다
2. 1회차 과제 구성
3. 제출 양식

파이썬 보강 과제 규칙

1회차 월말평가 결과와 개별 면담 내용을 종합하여 만든 문서입니다.
문제를 몇 개 풀었는지보다, 아래 규칙을 지켰는지가 훨씬 중요합니다.

1. 과제 규칙 (= 학습법)

규칙 1. 코드보다 먼저, 지문의 신호어를 찾는다

유형 1 처방 · 모든 과제의 첫 단계

문제는 도구를 안 알려주는 게 아닙니다. 다른 말로 알려줍니다.
아래 표를 옆에 두고 지문에서 신호어를 찾아 표시하세요.

지문에 이 말이 있으면	이 골격을 씁니다
모두, 각각, ~마다, 전부	반복문 (골격 공통)
합계, 총합, 누적, 모두 더한	누적
몇 개, 개수, 횟수, ~인 것의 수	카운팅
가장 큰/작은, 최대, 최소, 1등, 몇 번째	탐색
~인 것만, 조건에 맞는, 골라내시오, 제외한	필터링
~이면, ~일 때, ~인 경우, 단 ~는 제외	조건문
각 ~의 모든 ~ (2겹 구조), 2차원 리스트	중첩 반복문

반복문을 활용하시오 가 없어도 "주문 금액을 모두 더한"이라는 표현이 곧 반복문 + 누적 지시입니다.

규칙 2. 설계 주석을 먼저 쓴다 — 단, "왜"까지 쓴다

유형 1, 2 처방 · 이번 과제에서 가장 중요한 규칙

에디터를 열면 주석부터 씁니다. 코드는 그 아래를 채우는 것입니다.
그런데 지금까지처럼 "무엇을 하는지"만 쓰면 소용이 없습니다. 왜 그렇게 했는지를 반드시 함께 씁니다.

❌ 이렇게 쓰면 안됩니다

```
# 1. 합계를 담을 변수를 0으로 만든다
# 2. 리스트를 하나씩 돈다
# 3. 합계에 더한다
```

이건 완성된 코드를 한국어로 다시 읽은 것일 뿐입니다.
코드를 보면 어차피 아는 내용이라 아무 정보도 추가되지 않습니다.

✅ 이렇게 써야 합니다

```
# 1. [무엇을] total 변수를 0으로 만들고 시작한다.
#   [왜]      아직 아무것도 안 더한 상태를 숫자로 표현하면 0이기 때문.
#             1로 시작하면 실제 합보다 항상 1이 커진다.
#             그리고 리스트가 비어 있으면 반복문이 한 번도 안 돌아서
#             0이 그대로 반환되는데, 그게 문제가 요구하는 답과도 맞다.
#
# 2. [무엇을] orders를 for로 하나씩 꺼낸다.
#   [왜]      '모든 주문'을 처리해야 하는데 몇 개인지 모르니까,
#             개수를 몰라도 끝까지 도는 for가 맞다고 판단했다.
#
# 3. [무엇을] 꺼낸 값을 total에 더한다.
#   [왜]      total은 '지금까지의 합'을 계속 들고 있는 그릇이라서,
#             매 회차마다 갱신해야 마지막 회차에 전체 합이 남는다.
```

왜 이렇게까지 해야할까?

"왜"를 못 쓰면 그건 이해한 게 아니라 외운 것입니다.
유형 3(유형 암기)에서 벗어나는 유일한 방법입니다.
그리고 나중에 문제가 조금 바뀌었을 때, 바뀐 조건에 맞춰 조정할 수 있는 근거가 바로 이 "왜"입니다.

- total = 0 의 "왜"를 아는 사람은 **최댓값 찾기**에서 0으로 시작하면 안 된다는 걸 스스로 압니다.
- "왜"를 모르는 사람은 그냥 0을 쓰고 음수 데이터에서 틀립니다. (실제로 이번 과제 3번에 그 함정이 있습니다.)

금지 표현

아래 표현이 "왜" 칸에 들어가면 안됩니다.

- "그냥 그렇게 함"
- "원래 이렇게 하는 거라서"
- "강의에서 이렇게 했음"
- "아예 정답이라서"

"왜"를 한 줄도 못 쓰겠으면, 그건 문제를 못 푸는 게 아니라 아직 이해하지 못한 것입니다.
그때는 코드를 붙잡고 있지 말고 지문으로 돌아가거나 규칙 5(질문)로 넘어가세요.

규칙 3. 골격 4종은 백지에서 즉시 나와야 한다

외울 대상은 문제가 아니라 이 4개입니다.

골격	하는 일	시작값이 중요한 이유
1. 누적	합계·곱셈	0에서 시작 (아무것도 안 더한 상태)
2. 카운팅	조건 만족 개수	0에서 시작 (아직 안 썼음)
3. 탐색	최대/최소 + 그 인덱스	0이 아니라 첫 원소로 시작 (음수 함정)
3. 필터링	조건 만족 항목 모으기	빈 리스트에서 시작

시험 문제는 대부분 이 4개 중 하나이거나, 2개의 조합입니다.
새 유형처럼 보이는 문제도 뜯어보면 골격은 그대로이고 조건식만 바뀐 것입니다.

규칙 4. 실행 전에 결과를 손으로 예측한다

실행하기 전에, 반복 회차별 변수 값을 표로 적으세요.

회차	꺼낸 값	조건 판정	total
1	12000	-	12000
2	8500	-	20500
3	30000	-	50500

예측과 실재가 다르면, 거기가 정확히 본인이 잘못 알고 있는 지점입니다.
표 없이 실행만 반복하면 "왜 되는지 모르는데 답은 맞는" 상태가 되고, 그 상태로는 시험장에서 응용이 안 됩니다.

규칙 5. 30분 넘게 막히면 무조건 질문한다

한 문제를 3시간 붙잡는 건 성실함이 아니라 시간 낭비입니다.
타이머를 켜고, 올리면 손을 떼고 질문(ai, 강사)하세요. 질문할 때 아래 3가지를 함께 적습니다.

1. 내가 이해한 문제 요구사항
2. 내가 시도한 코드 (틀려도 그대로)
3. 어디서 어떻게 막혔는지

규칙 6. 이해가 안 돼도 일단 넘어간다

"이해를 못 하면 잘 못 넘어간다"는
성실해 보이지만, 실제로는 진도를 가장 크게 갇아먹는 습관입니다.

프로그래밍은 앞 개념을 100% 이해해야 다음으로 가는 구조가 아닙니다.
오히려 뒤에 나오는 예제를 보고 나서 앞 개념이 이해되는 경우가 훨씬 많습니다.
80% 이해됐으면 표시만 해두고 넘어가세요. 붙잡고 있는 사이 밀린 진도가 다음 달 과락이 됩니다.

규칙 7. 문제를 하나 변형해서 고민해본다

5문제를 다 푼 사람은 그중 하나를 골라 조건을 바꿔 새 문제를 직접 고민해보세요.

- 예: "주문 금액의 총합" => "**10,000원 이상인 주문만** 골라 총합"

- 예: "최고 기온의 인덱스" => "최저 기온의 인덱스, 단 동점이면 뒤쪽"

2. 1회차 과제 구성

번호	함수명	골격	
01	sum_order_total	1. 누적	
02	count_late_days	2. 카운팅	
03	find_hottest_day	3. 탐색	
04	filter_low_stock	4. 필터링	
05	analyze_scores	5. 조합 (중첩 + 카운팅 + 탐색)	

3. 제출 양식

배포된 problem01.py ~ problem05.py 파일에 이미 양식이 들어 있습니다. 양식 칸을 지우지 말고 채워서 제출하세요.

```
# =====
# [ 1 ] 신호어 찾기
# =====
# 지문에서 찾은 신호어 :
# 선택한 골격          :
# 그렇게 판단한 이유  :

# =====
# [ 2 ] 설계 - 무엇을 / 왜
# =====
# 1. [무엇을]
#   [왜]
# 2. [무엇을]
#   [왜]
# 3. [무엇을]
#   [왜]

# =====
# [ 3 ] 손 추적표 (실행 전에 작성)
# =====
# 회차 | 꺼낸 값 | 조건 판정 | 결과 변수
# 1    |          |          |

# =====
# [ 4 ] 회고
# =====
# 막힌 지점          :
# 예측과 실제 차이   :
```

정답만 적힌 제출물보다, 틀렸지만 설계와 "왜"와 막힌 지점이 적힌 제출물이 훨씬 좋은 제출물입니다.
전자는 피드백할 게 없지만, 후자는 정확히 어디를 고쳐야 하는지 보이기 때문입니다.